



VIT[®]

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

School of Computer Science and Engineering

BCSE305L – Embedded Systems

Module ~ IV Programming Tools

Code Optimization

Presentation by

Dr.K.Ragavan

Assistant Professor Senior Grade I

Department of IOT

School of Computer Science and Engineering

VIT University, Vellore

Dr.K.Ragavan, SCOPE, VIT Vellore

Code Optimization

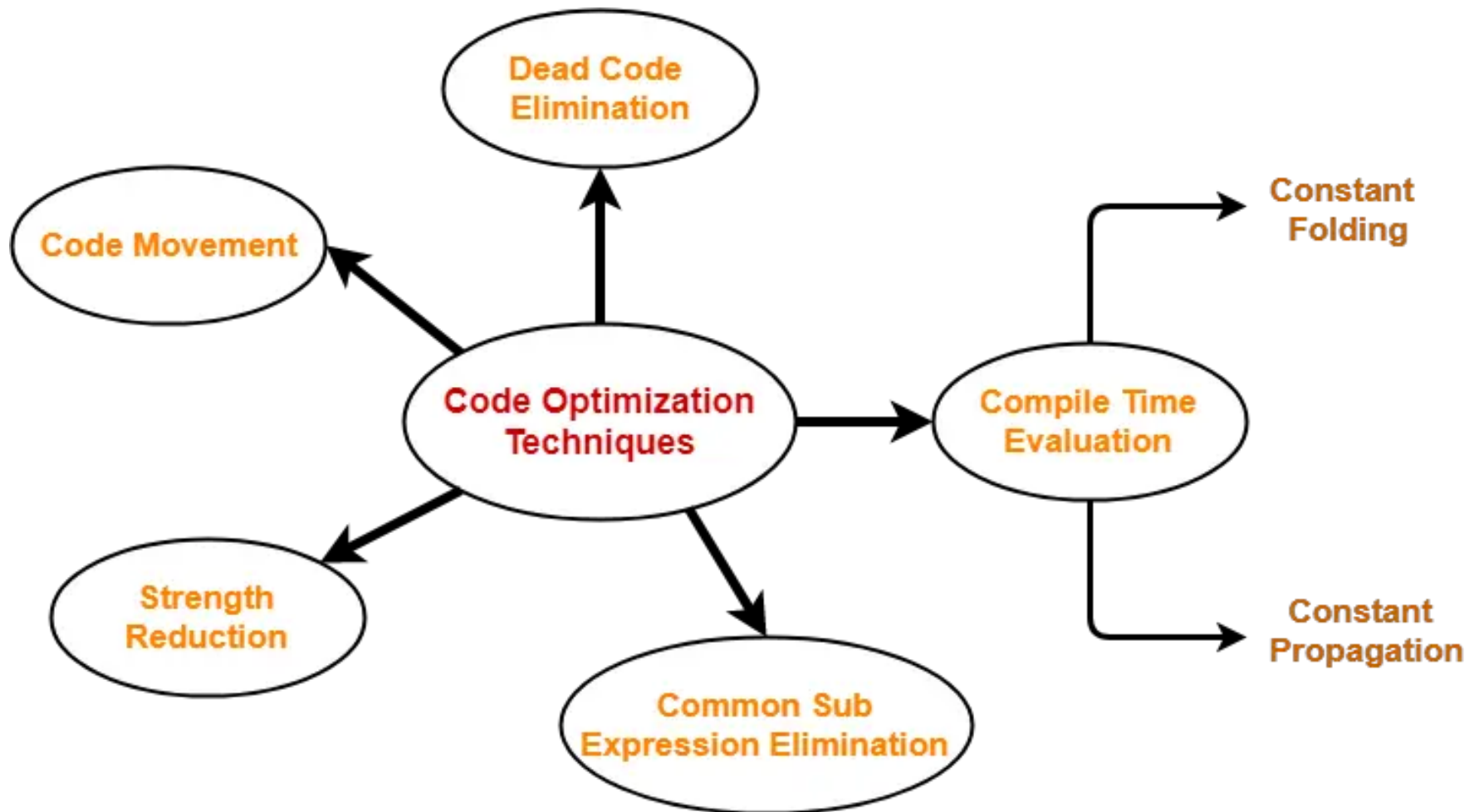
- The idea is that code executed on an embedded system has to be optimised both to utilise available limited resources in way that meets all system requirements.
- In practical terms, embedded code is written to make optimum use of processor or compiler features, as well as managing the memory or power requirement.

Definition

Code Optimization is a technique which tries to improve the code by eliminating unnecessary code lines and arranging the statements in such a sequence that speed up the program execution without wasting the resources.

Advantages:-

- Executes faster
- Efficient memory usage
- Yields better performance



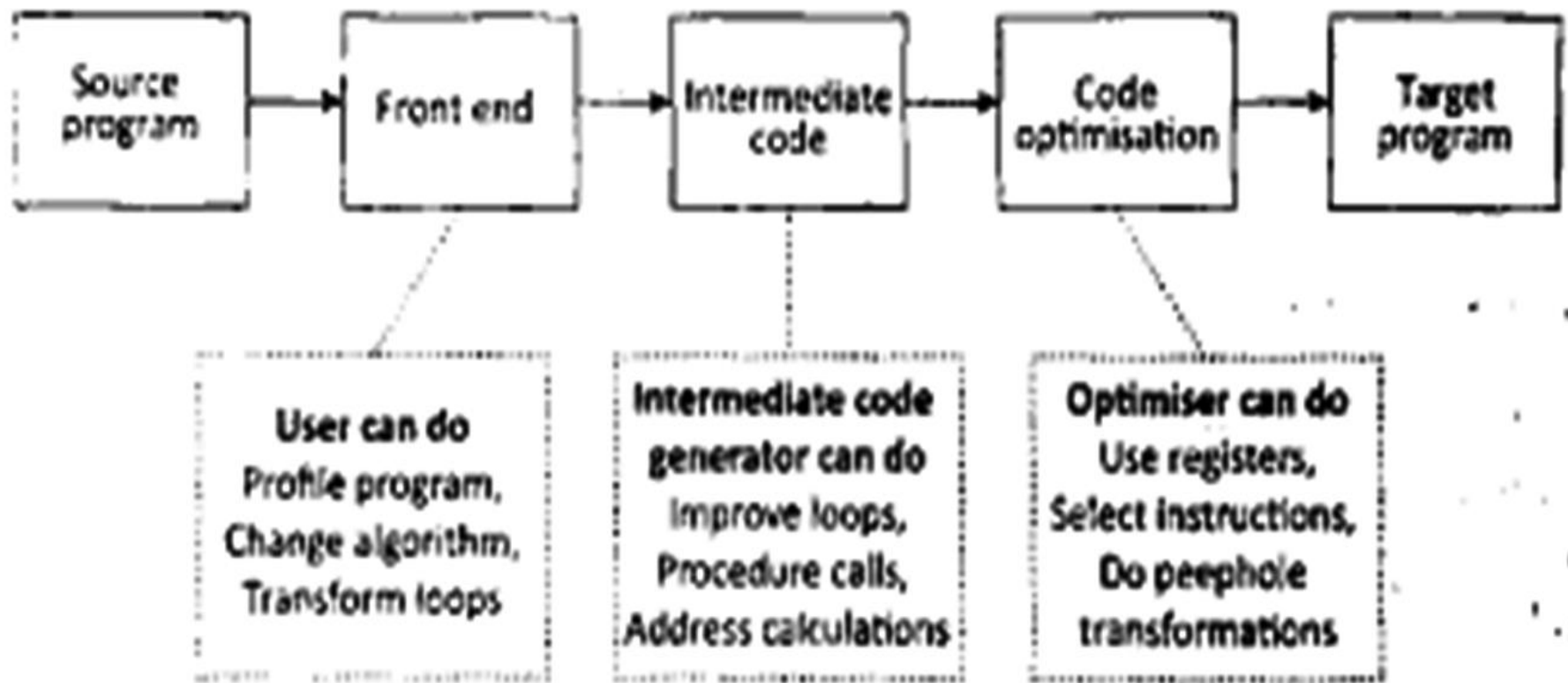
To produce an efficient code

Two issues in code optimization is

- Semantic equivalence of the source program must not be changed
- It must be achieved without changing the algorithm of the program
- Program becomes inefficient because of programmer or due to compiler transformation
- Now where code optimization should be applied – before code generation or after code generation

Compiler phases

- Front end (3 phases)
- back end



Compile Time Evaluation

- we optimize the code at the compile time – two techniques
 - constant folding
 - constant propagation
- **Constant Folding** – technique of evaluating an expression whose operands are known to be constants at compile time itself

Example: $\text{Length} = (22/7)*d$

- Above $22/7$ is a constant expression
- So first solve the constant expression before and keep the constant value – so called folding the constant and place the value at compile time since evaluated at compile time and keep the value of reuse

① Compile Time Evaluation

i) Constant Folding:

It refers to a technique of evaluating the expressions whose operands are known to be constant at compile time itself.

Example:- $\text{length} = (2217) * d$

ii) Constant Propagation:-

In constant propagation, if a variable is assigned a constant value, then subsequent use of that variable can be replaced by a constant as long as no intervening assignment has changed the value of the variable.

Example – finding area of circle

Example:-

$$\pi = 3.14$$

$$r = 5$$

$$\text{Area} = \pi * r * r$$

Here, the value of π is replaced by 3.14 and r by 5, then computation of $3.14 * r * r$ is done during compilation.

So values of variables are directly substituted at the place of reference or useage and executed at compile time itself

⑪ Common sub-expression elimination :-

The common sub-expression is an expression appearing repeatedly in the code which is computed previously. This technique replaces redundant expression each time it is encountered.

Example:-

$T_1 = 4 * i$
 $T_2 = a[T_1]$
 $T_3 = 4 * j$
 $T_4 = 4 * i$
 $T_5 = n$
 $T_6 = b[T_4] + T_5$

Before Optimization

$T_1 = 4 * i$
 $T_2 = a[T_1]$
 $T_3 = 4 * j$
 $T_5 = n$
 $T_6 = b[T_4] + T_5$

After Optimization

- Above is the 3 address code
- So $4 * i$ is not executed second time for T_4 and value of T_1 is taken for T_4
- So no redundant expression in the optimized code

III Code Movement:-

It is a technique of moving a block of code outside a loop if it won't have any difference if it is executed outside or inside the loop.

Example:-

```
for (int i=0; i<n; i++)  
{  
    x = y+z;  
    a[i] = 6*i;  
}
```

Before Optimization

```
x = y+z;  
for (int i=0; i<n; i++)  
{  
    a[i] = 6*i;  
}
```

After Optimization

④ Dead Code Elimination:-

Dead Code Elimination includes eliminating those code statements which are either never executed or unreachable or if executed their output is never used.

Example:-

```
i = 0
if (i == 1)
{
    a = x + 5;
}
```

Before Optimization

```
i = 0
```

After Optimization

⑤ Strength Reduction:

It is the replacement of expressions that are expensive with cheaper and simple ones.

Example:

$$B = A * 2$$

Before Optimization

$$B = A + A$$

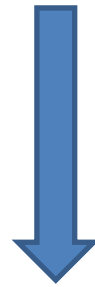
After Optimization

Replace an expression having more priority with less priority expression

- **Strength reduction** used to eliminate the multiplication from the induction variable computation.
- Strength reduction is a compiler optimization where **expensive operations are replaced with equivalent but less expensive operations.**
- The classic example of strength reduction converts **"strong" multiplications** inside a loop into **"weaker" additions**

Strength Reduction

```
c = 7;  
for (i = 0; i < N; i++)  
{  
    y[i] = c * i;  
}
```



```
c = 7;  
k = 0;  
for (i = 0; i < N; i++)  
{  
    y[i] = k;  
    k = k + c;  
}
```

Loop Optimization

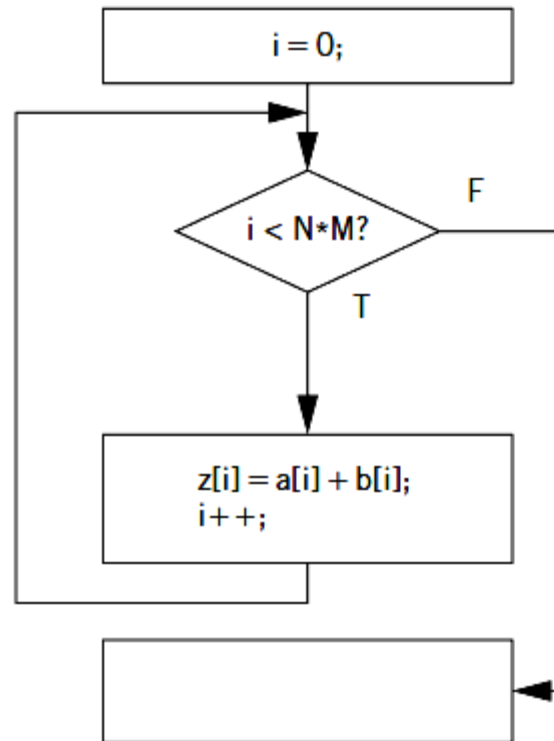
- Loops are important targets for optimization because programs with **loops** tend to spend a **lot of time** executing those loops.
- There are three important techniques in optimizing loops:
 - **Code motion**
 - **Induction variable elimination**
 - **Strength reduction.**

➤ **Code motion** lets us move unnecessary code out of a loop.

➤ Consider the following loop:

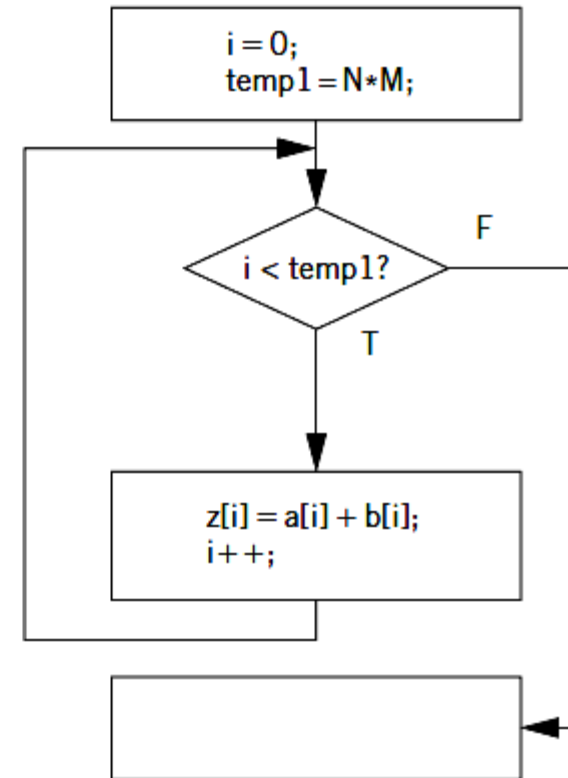
```
for (i = 0; i < N*M; i++)  
{  
    z[i] = a[i] + b[i];  
}
```

We optimize the
code in inner loop



Dr.K.Ragavan, SCOPE, VIT Vellore

Before



After

- **An induction variable** is a variable whose value is derived from the loop iteration variable's value.

```
int a[SIZE];  
int b[SIZE];
```

```
void f (void)  
{  
    int i1, i2, i3;  
  
    for (i1 = 0, i2 = 0, i3 = 0; i1 < SIZE; i1++)  
        a[i2++] = b[i3++];  
    return;  
}
```



```
int a[SIZE];  
int b[SIZE];
```

```
void f (void)  
{  
    int i1;  
  
    for (i1 = 0; i1 < SIZE; i1++)  
        a[i1] = b[i1];  
    return;  
}
```

Loop invariant method

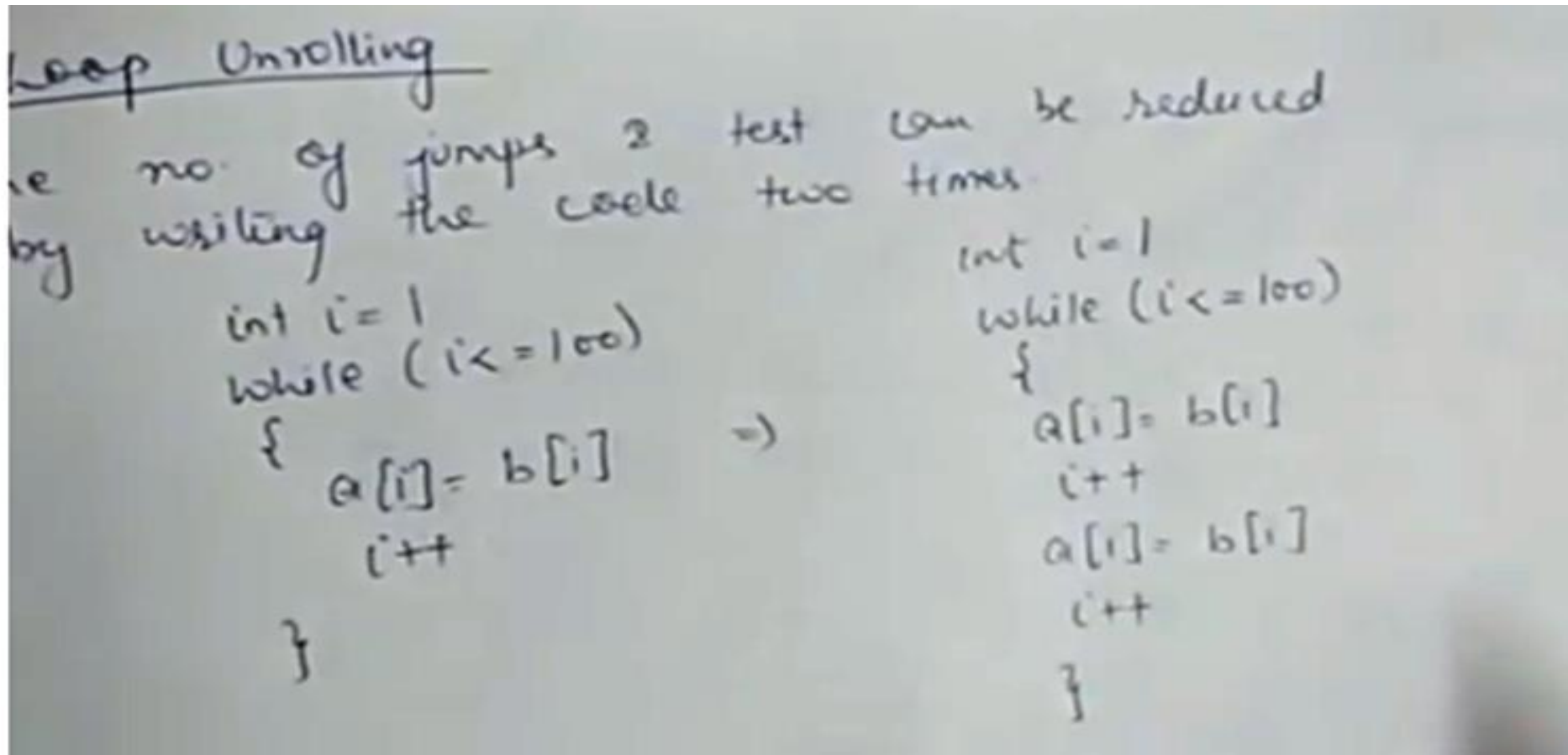
- Computation inside the loop is avoided and thereby computation overhead on compiler is avoided

③ Loop invariant - the computation inside the loop is avoided and thereby computation overhead on compiler is avoided

```
for i = 0 to 10 do begin  
    k = i + a/b  
    ...  
end
```

Loop unrolling

- **Number of jumps and tests** can be reduced by writing the code two times



So we reduce the no of steps to some extent by repeating the operation in the body

Loop fusion or loop jamming

- Several loops are merged to one loop

(5) Loop Fusion / Loop Jamming
Several loops are merged to one loop.
 $\text{for } i = 1 \text{ to } n \text{ do}$
 $\text{for } j = 1 \text{ to } m \text{ do}$
 $a[i, j] = 10$

$\rightarrow$

$\text{for } i = 1 \text{ to } n \times m \text{ do}$
 $a[i] = 10$